Calibrated Lightweight Ensembles for Predictive AI Evaluation

Yuhang Ling **Tao Zhang** **Hengzhi Mu** **Ruixi Dai**
School of Software, Shandong University

Abstract

We present a lightweight predictor for the Predictive AI Evaluation Competition at NeurIPS 2026. A constant mean, Ridge regression, and ExtraTrees regression combine public system attributes, structured item features, and hashed text. A log-odds intercept calibrates their probabilities; pseudo-count and hierarchical logistic updates incorporate own-configuration and peer labels. The final base uses 150,000 sampled responses from 105 public binary benchmarks. In public development on 41 held-out benchmarks, benchmark-excluded calibration reduces Brier area under the learning curve (ALC) from 0.196028 to 0.191748 with own labels and from 0.193754 to 0.190275 with a peer. Gains concentrate at low label budgets and vary across benchmarks. Our best recorded formative score was a Brier ALC of 0.139514 (Codabench submission 1020550).

1 Introduction and task

Predictive evaluation estimates an AI system's response probabilities from its configuration, the task description, and limited observed outcomes. Its central challenge is transferring across benchmarks: item-level patterns learned on public tasks must be combined with evidence about the success level of a new task collection. This setting calls for both a useful cold-start predictor and an effective update from the first few revealed labels.

Our method separates these roles. Sparse lexical regression and structured trees supply item-level predictions. A global intercept adjusts their probability level, an own-configuration update estimates local success, and a hierarchical model shares same-benchmark evidence across configurations. This decomposition keeps the supervised model fixed while allowing small, inexpensive adaptation problems at evaluation time.

For input $x = (s, i)$ and evidence $\mathcal{L}_k$, the program returns a success probability $p(x \mid \mathcal{L}_k)$. Per-pair budgets are $k \in \{0, 1, 3, 7, 15, 31\}$. With scored responses R_g and binary outcomes y_r for system–benchmark pair g, the metric is

$$B_{g,k} = \frac{1}{|R_g|} \sum_{r \in R_g} (p(x_r \mid \mathcal{L}_k) - y_r)^2, \tag{1}$$

$$A_g = 0.1B_{g,0} + 0.2(B_{g,1} + B_{g,3} + B_{g,7} + B_{g,15}) + 0.1B_{g,31}. \tag{2}$$

The score averages A_g equally across pairs [1]. Identical inputs reuse a prediction, while all response trials contribute to error. The weights arise from trapezoidal integration on $\log_2(k + 1)$: half of the total weight lies at budgets zero, one, and three. Our design targets this low-budget regime through calibrated priors, explicit shrinkage, and configuration-aware sharing.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

2 Public data and feature construction

2.1 Filtering, sampling, and weighting

The training source is the public Measurement Data Bank release [2]. Filtering yields 105 binary benchmarks and 55,047,908 candidate responses; the final base ensemble is fitted to a 150,000-row sample. The loader checks entity joins and response identifiers, retains public releases, and excludes a benchmark when its response scale includes invalid or non-binary labels. It also excludes `bigfinancebench`, marked upstream as evaluation-only. Predictive inputs come from response-linked subject and item tables, using the public configuration, task text, static attributes, and interactors.

Sampling precedes text attachment. With seed 0, the implementation allocates an equal initial quota to every subject–benchmark group, samples up to that quota, then fills unused capacity from the remaining rows. Text is joined only for the selected observations. This ordering makes the sample size, rather than the full response count, determine the number of expanded text records. It is useful for collections with substantial size imbalance: `monkey_power_laws` alone contributes 28,780,000 candidate responses, approximately 52.3% of the retained pool.

The fitting weights balance the represented groups after sampling. For N sampled observations, G groups, and group size N_g, the weight and the resulting training mean are

$$w_j = \frac{N}{G N_{g(j)}}, \qquad \mu = \frac{\sum_j w_j y_j}{\sum_j w_j} = \frac{1}{G} \sum_{g=1}^{G} \frac{1}{N_g} \sum_{j \in g} y_j. \tag{3}$$

Every group therefore receives total weight N/G, and the mean weight is one. Quota sampling provides coverage; inverse-count weighting gives the regression objective the same pair-balanced emphasis as evaluation. Internal identifiers support joins, sampling, and weighting; the learned feature vectors use the public attributes below.

2.2 Visible attributes and two feature branches

The eight accepted subject fields are normalized name, provider, release date, access date, harness, harness version, reasoning effort, and extra configuration attributes. Preprocessing maps missing values and exact null-like strings to empty values, strips surrounding whitespace from other strings, and parses extensions as JSON dictionaries or semicolon-delimited key–value strings. Explicit whitelists retain static attributes such as language, task category, context length, and model configuration.

Table 1: Input representations and their roles in the final predictor.

Component	Representation and role
Structured item	22 measurements of length, character composition, textual indicators, domain, and length bin; retained static attributes.
Structured subject	Public categorical fields, parsed log model size, size availability, and reasoning-effort availability.
Text	Signed word unigram/bigram hashing of subject and item text into 2^{16} dimensions; L2 normalization.
Evidence grouping	Anonymous benchmark ID and normalized public subject identity select labels for adaptation.

`DictVectorizer` produces 1,444 structured columns in the fitted artifact. Ridge combines these with the hashed text; ExtraTrees uses the structured columns [4]. The two branches capture complementary patterns: lexical coefficients add contributions across words and attributes, while trees model interactions among categorical and numeric features. Item content and the item text passed to the hasher are limited to 12,000 characters. Access date enters the structured branch and subject identity; hashed subject text includes release date. These lexical and handcrafted features support consistent encoding of newly encountered task text without fitting a new vocabulary. Full fields and preprocessing rules appear in Appendix A.

3 Base ensemble and global calibration

3.1 Weighted regression ensemble

The base learner combines a weighted constant mean, Ridge regression, and ExtraTrees regression [3, 4]. Let z_j concatenate the structured and hashed features. Ridge fits an intercept c and coefficients β by minimizing

$$\sum_j w_j(y_j - c - z_j^\top \beta)^2 + 100\|\beta\|_2^2. \tag{4}$$

ExtraTrees models nonlinear relationships in the structured vector. Its randomized splits and minimum leaf size pool observations into local response-rate estimates, while the linear branch represents sparse lexical associations. Both learners use the same group weights. The constant component supplies additional shrinkage toward the pair-weighted mean, $\mu = 0.5743583606802852$.

Writing $C(p) = \mathrm{clip}(p, 0.001, 0.999)$, the combination is

$$p_0(x) = C(0.25C(\mu) + 0.375C(f_{\mathrm{R}}(x)) + 0.375C(f_{\mathrm{T}}(x)))\,. \tag{5}$$

The individual regressors' outputs are clipped before mixing, followed by clipping of the mixture. This keeps the subsequent log-odds operations finite and limits the contribution of extreme regression outputs.

The weights also express a simple shrinkage structure: 75% of the prediction comes from the equal-weight average of Ridge and ExtraTrees, and 25% comes from the global mean. For a fixed fitted model, the constant branch cancels in differences between targets. The ensemble therefore scales differences from the two-regressor average by 0.75 while anchoring the overall level at the pair-balanced mean.

Table 2: Selected model and adaptation parameters.

Component	Configuration
Ridge	$\alpha = 100$; `solver=lsqr`; fitted intercept; tolerance 10^{-4}.
ExtraTrees	80 trees; depth at most 12; minimum leaf size 20; seed 42; `max_features=1.0`.
Global calibration	Slope $a = 1$; $b = -0.5796672067792444$; penalty $0.001b^2$; all budgets.
Own-label update	Pseudo-count strength $\tau = 2$; item-retention weight $\rho = 0.5$.
Hierarchical update	Task precision 0.5; configuration precision 2; shared mixture weight 0.5.

3.2 A one-parameter probability-level correction

The calibrated probability is

$$p(x) = C(\sigma(\mathrm{logit}(p_0(x)) + b))\,, \qquad \sigma(t) = (1 + e^{-t})^{-1}. \tag{6}$$

Before boundary clipping, the transform multiplies odds by $e^b = 0.5601$. It maps base probabilities 0.25, 0.50, and 0.75 to approximately 0.1573, 0.3590, and 0.6269. Thus the negative intercept reduces probability levels while retaining ordering; probabilities below 0.5 also decrease. Targets and acquired examples receive the same transform, giving the adaptation rules a common probability scale.

For E public out-of-benchmark prediction episodes, we fit

$$L(b) = \frac{1}{E}\sum_{e=1}^{E}\frac{1}{N_e}\sum_{j=1}^{N_e}[C(\sigma(\mathrm{logit}(p_{0,ej}) + b)) - y_{ej}]^2 + 0.001b^2. \tag{7}$$

Episodes have equal weight and responses have equal weight within each episode. L-BFGS-B optimizes $b \in [-3, 3]$ with at most 100 iterations and `ftol`$= 10^{-12}$. The final fit uses 758 episodes from 47 public benchmarks and has penalized fitting loss 0.2337508337. Fixing the slope gives a probability-level correction with one fitted degree of freedom. Section 6 evaluates the development calibration protocol and the frozen confirmation intercept specified in Table 3.

4 Adaptation using revealed labels

4.1 Own-configuration pseudo-count update

For each target, the predictor selects labels with the same benchmark ID and the same eight normalized public subject fields. Including harness, version, reasoning effort, and dates distinguishes configurations of the same named model. Let $e_j = p(s, i_j)$ be the calibrated probability of acquired item j, y_j its outcome, and n the matching label count. For $n > 0$, define

$$\bar{p} = \frac{1}{n}\sum_{j=1}^{n} e_j, \qquad m = \frac{\tau\bar{p} + \sum_{j=1}^{n} y_j}{\tau + n}, \qquad \tau = 2. \tag{8}$$

$$q(x) = \rho\sigma(\mathrm{logit}(C(p(x))) - \mathrm{logit}(C(\bar{p})) + \mathrm{logit}(C(m))) + (1-\rho)m, \quad \rho = 0.5. \tag{9}$$

For $n = 0$, $q(x) = p(x)$. The prior is centered on the acquired examples' mean base prediction. This accounts for the difficulty of the observed items when converting their outcomes into a success-level update. The empirical label mean receives weight $n/(n+2)$: one third for one matching label, three fifths for three, and $31/33$ for 31. The matching count n need not equal the per-pair budget k.

The logistic shift expresses target difficulty relative to $\bar{p}$ and moves its center to m. Away from the clipping bounds, a target with $p(x) = \bar{p}$ is mapped to m in both mixture components. For other targets, $\rho = 0.5$ retains half of the shifted item-level variation. The pseudo-count update estimates a shrinkage success rate, then shifts item difficulty to that rate.

4.2 Hierarchical sharing across configurations

When another public configuration has revealed labels on the target benchmark, all same-benchmark evidence is additionally used to fit a shared task offset u and configuration offsets v_k. With $z_j = \mathrm{logit}(C(e_j))$ and configuration $k(j)$, the objective is

$$\mathcal{J}(u, v) = \sum_j [\log(1 + \exp(z_j + u + v_{k(j)})) - y_j(z_j + u + v_{k(j)})] + \frac{0.5}{2}u^2 + \frac{2}{2}\sum_k v_k^2. \tag{10}$$

The base logit is a fixed offset, so fitting focuses on residual probability shifts. The task term captures a benchmark-wide effect; configuration terms absorb differences among systems. Configuration precision is four times task precision, favoring a shared adjustment when each configuration has few observations. Labels already explained by a high or low base probability exert correspondingly smaller residual pressure on these offsets.

For example, at the initial zero offsets, a success with calibrated probability 0.9 contributes a logistic residual of -0.1, while a success with probability 0.1 contributes -0.9. The latter pushes the task and configuration offsets more strongly. Sharing thus transfers outcomes relative to their predicted difficulty, allowing peer evidence to reflect both the strength of the observed configuration and the common benchmark effect.

Newton iteration solves the penalized logistic system for at most 25 steps, clips each coordinate update to $[-2, 2]$, and stops when the largest absolute un-clipped step is below 10^{-8}. For target configuration s,

$$h(x) = \sigma(\mathrm{logit}(C(p(x))) + \hat{u} + \hat{v}_s), \tag{11}$$

$$p_{\mathrm{final}}(x \mid \mathcal{L}_n) = \begin{cases} C(0.5C(q(x)) + 0.5h(x)), & \text{with other-configuration evidence,} \\ C(q(x)), & \text{otherwise.} \end{cases} \tag{12}$$

A target configuration with no labels has $\hat{v}_s = 0$ and can still receive the task adjustment learned from peers. The final mixture combines a target-specific pseudo-count estimate with the hierarchical estimate. Each call constructs its evidence groups from the supplied `labeled` argument. The platform's default random acquisition supplies nested label sets, and adaptation uses the labels belonging to the current benchmark.

5 Experimental design and model selection

5.1 Public benchmark-held-out simulation

Episodes require at least 80 distinct items per subject–benchmark group. A seeded group-specific shuffle assigns the first half to acquisition and the remainder to evaluation, capped at a specified number of items. One response is sampled per acquired item, up to 31; all recorded trials of selected evaluation items are retained. Acquisition prefixes give budgets 0, 1, 3, 7, 15, and 31, with a fixed target set across budgets.

Development reuses 698 saved episodes from three historical files with their original construction settings (Appendix C.2); it is not one newly sampled pool. The validation base fits 123,060 rows, excluding all 41 development and six confirmation benchmarks. The 36 selection tasks supply 437 episodes and use leave-one-benchmark-out intercepts. The other five tasks—`aegis`, `babilong`, `bird_sql`, `real_webagents`, and `researchcodebench`—supply 261 episodes and all use $b = -0.4100424649$, fitted on the complete 437-episode pool. They have earlier V3 development history but do not enter this selection-stage intercept fit; all five enter the final 758-episode refit.

5.2 Evidence scenarios and calibration candidates

The own-label scenario supplies the target group's acquired responses. The peer scenario adds the corresponding-budget labels of one other same-benchmark subject group, selected deterministically when available. Configuration identities determine whether the evidence is pooled within the same identity or passed to the hierarchical sharing rule. Both scenarios use equal-episode ALC; this weights each episode's complete learning curve equally while retaining all recorded target trials within that episode.

Eight calibration specifications vary three factors: intercept-only versus slope/intercept, regularization 0.001 versus 0.01, and application before adaptation at all budgets versus only when benchmark evidence is absent. Selection retains candidates with non-degrading ALC in both scenarios and an average improvement exceeding 0.0002, then minimizes average scenario ALC. The selected specification is the all-budget intercept with regularization 0.001.

At regularization 0.001, this specification achieves mean scenario ALC 0.191012, compared with 0.193170 for the intercept applied only without evidence and 0.192233 for all-budget slope/intercept calibration. Raising the all-budget intercept penalty to 0.01 yields 0.191274. These comparisons favor keeping the correction active during adaptation and retaining the simpler one-parameter transform. Appendix C reports all eight candidates.

Table 3: Training and evaluation stages. Local ALCs use the development or frozen confirmation estimators; the final intercept is refitted on all 758 public episodes.

Stage	Public data	Estimator and role
Development	41 benchmarks; 698 episodes	Held-out base; 36 task-specific leave-outs and five shared-intercept tasks.
Confirmation	6 benchmarks; 60 episodes	Frozen $b = -0.4100424649$, fitted on 437 selection episodes.
Final calibration	47 benchmarks; 758 episodes	$b = -0.5796672068$, fitted using all development and confirmation predictions.
Final base	105 benchmarks; 150,000 rows	Full-public ensemble used in the exported submission.

Confirmation episodes use seed 1001029, an evaluation cap of 60 items, and at most 12 randomly selected groups per benchmark. The six confirmation benchmarks are `ceval`, `mathconstraint`, `jailbreakbench`, `helm_bold`, `swe_smith`, and `medhelm_med_qa`. They provide 11, 12, 10, 12, 3, and 12 episodes respectively. The selected specification and frozen intercept are evaluated on these tasks before the final calibration refit. Export combines that refitted intercept with the 150,000-row full-public base and the fixed own-label and sharing parameters.

6 Public validation results

6.1 Development gains by budget and benchmark

These results use the 123,060-row validation base with 47 benchmarks held out, rather than the submitted 150,000-row base. Table 7 compares calibration under the protocol in Section 5. Development ALC decreases by 0.004279 with own labels (2.18% relative) and by 0.003479 with a peer (1.80%). These are episode-weighted aggregate changes across the 41 development benchmarks.

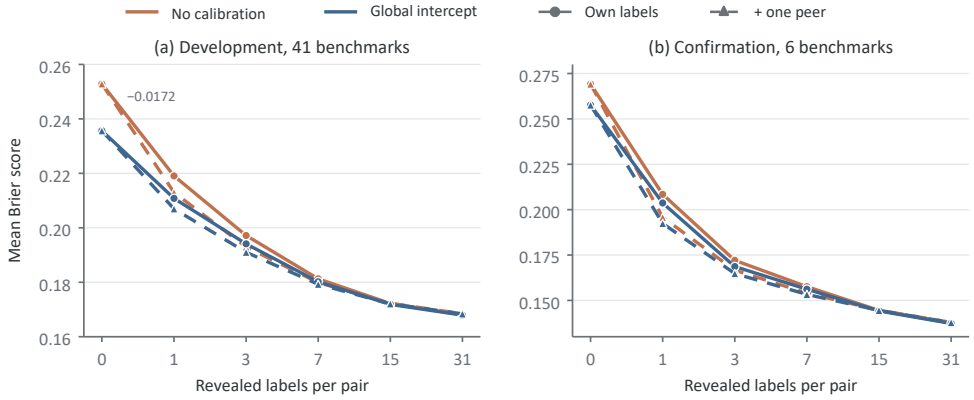


Figure 1: Brier error by label budget on development (left) and confirmation (right). Colors distinguish calibration; circles use own labels and triangles add one peer. Gains are largest at low budgets.

In Figure 1, own-label development Brier error falls from 0.252865 to 0.235650 at budget zero and from 0.219054 to 0.210813 at budget one. Their ALC-weighted contributions total 0.003370, or 78.7% of the own-label net reduction. By budget 31, the difference is 0.000106. The decreasing gap is consistent with local labels progressively supplying the success-level correction initially provided by global calibration.

Across benchmarks, own-label ALC improves on 22 and increases on 19. The 120 `bird_sql` episodes improve from 0.114161 to 0.098920, contributing $120/698 \times 0.015241 = 0.002620$, or 61.2% of the net reduction. Increases include `wikihow_agent` (0.081026 to 0.107501) and `helm_bbq` (0.122097 to 0.138990). Thus much of the aggregate gain comes from a few tasks (Figure 2 in Appendix D). `bird_sql` belongs to the five tasks excluded from selection-stage calibration fitting.

With benchmarks weighted equally, own-label ALC decreases from 0.209268 to 0.207620 and peer ALC from 0.207308 to 0.206002. The contrast shows how the episode distribution affects the aggregate gain: `bird_sql` represents 17.2% of development episodes.

Adding one same-benchmark subject group's labels lowers calibrated development ALC by 0.001473; this comparison includes additional evidence and identity-based pooling, as well as hierarchical sharing. Relative to own labels, Brier error improves by 0.003834 at budget one and 0.003078 at budget three, but increases by 0.000233 at fifteen and 0.000454 at thirty-one. Peer evidence contributes most when direct labels are sparse.

6.2 Confirmation and task-level variation

The frozen confirmation intercept improves ALC by 0.003149 with own labels and 0.002369 with a peer. We resample the six complete benchmark clusters 5,000 times with seed 901029, summing episode-level differences and dividing by the sampled episode count. The 2.5th and 97.5th percentiles give Table 8's intervals; all include zero.

Calibration improves three confirmation benchmarks and increases error on three, in both scenarios. With own labels, `helm_bold` improves from 0.105294 to 0.078077, whereas `medhelm_med_qa` increases from 0.126305 to 0.139335. The `helm_bold` change contributes $12/60 \times 0.027217 = 0.005443$ before other task changes are combined, exceeding the aggregate reduction. Appendix D gives the complete per-task and per-budget results.

7 Online formative results

Codabench submission 1020550 obtains Brier ALC 0.139514 in a formative evaluation comprising nine subject–benchmark groups. Applying Equation 2 to the six group-mean Brier scores gives 0.1395135333, matching the displayed rounding. Table 4 reports the budget curve; Appendix E lists the nine recorded V10 formative submissions.

Table 4: Submission 1020550: mean Brier error across nine groups at each label budget.

Labels per pair	0	1	3	7	15	31
Mean Brier	0.208803	0.143657	0.140168	0.123155	0.122139	0.128092
ALC weight	0.1	0.2	0.2	0.2	0.2	0.1

From per-pair budget 0 to 31, mean error falls by 0.080711. The change from budget 0 to 1 contributes 0.065146 of this decrease (0.208803 to 0.143657); available evidence can include other configurations on the same benchmark. Error reaches its minimum, 0.122139, at budget 15 and rises by 0.005953 at budget 31.

Against the constant-0.5 reference (ALC 0.25), the nine-submission mean 0.184069 corresponds to a reduction of 0.065931, or 26.4%. The selected sample's reduction is 0.110486, or 44.2%. These comparisons summarize the complete predictor, including its cold-start probabilities and label updates.

At budget 31, mean Brier error rises to 0.128092, a 0.005953 increase from budget fifteen. The scored items are fixed across budgets within this submission, so the change reflects the predictor's response to the enlarged evidence set. The pseudo-count estimate gives greater weight to the observed outcomes as n grows, and hierarchical offsets also change when further same-benchmark labels arrive.

Across the nine recorded formative submissions, the mean ALC is 0.184069, the median is 0.189484, and the sample standard deviation is 0.026420. Scores range from 0.139514 to 0.218715. The selected submission is 0.044555 below the mean.

Within one submission, all budgets use the same evaluation responses. Across the nine records, the platform resamples subjects, benchmarks, and items [1]; Appendix E lists their IDs and scores.

8 Implementation, reproducibility, and discussion

8.1 CPU inference and call-local adaptation

The Python entry point `predict(input, labeled)` returns a finite native float. It loads the model on its first call and uses one numerical thread and tree-inference worker. The 3,467,137-byte ZIP bundles the predictor, runtime modules, metadata, and requirements. Feature construction, regression, and adaptation run locally on a CPU.

Each call builds evidence groups from `labeled`; shared state comprises the model and an 8,192-entry feature cache. With no matching same-benchmark evidence or an empty benchmark ID, prediction uses the calibrated base. Own-configuration labels trigger the pseudo-count update; other-configuration labels additionally trigger the equal-weight hierarchical mixture.

With M same-benchmark labels and K configuration coordinates, each dense Newton iteration costs approximately $O(MK^2 + K^3)$. Adaptation fits offsets while retaining the public-data encoders and regressors (Appendix B).

8.2 Environment and reproducible operations

The code and reproduction materials are available in the public project repository: `https://github.com/KyrieLYH/2026-NeurIPS-Competition`.

The recorded environment uses Python 3.10.21 and scikit-learn 1.7.2; Appendix G lists all dependency versions. Interface checks cover six budgets, deterministic outputs, unchanged inputs, benchmark grouping, long text, JSON attributes, missing fields, and label adaptation.

Table 5: Reproduction tools. Full commands appear in Appendix G.

Script in `tools/`	Operation
`verify_bundle.py`	Check the file manifest and original ZIP members.
`check_submission.py`	Run isolated loading and interface checks.
`fit_selected_calibration.py`	Refit the intercept from 758 cached public episodes.
`train_selected_base.py`	Sample public tables and train the fixed base recipe.
`package_submission.py`	Assemble the runtime files into a submission ZIP.

Cached public predictions support intercept refitting. Base retraining accepts raw tables or a feature-attached sample and saves new sample identifiers; packaging assembles the scoring runtime.

LLM-assisted development. We used OpenAI Codex to assist with code implementation, experiment execution, validation, and documentation.

8.3 Technical limitations and next steps

Local ALCs evaluate the 123,060-row base excluding 47 benchmarks, rather than the submitted combination of unchanged V3 150,000-row full-public weights and $b = -0.5796672068$, refitted on 758 validation-base episodes. Lexical features and the 12,000-character limit restrict semantic and long-context information. Peer sharing depends on overlap between configurations' residual task effects; budget-level and benchmark-level results show where it helps.

The historical 150,000-row sample manifest and pinned data revision are missing; retraining implements the fixed recipe without guaranteeing byte-identical weights. Future runs should save revisions, sample identifiers, and episodes, and test adaptation with more peer configurations.

8.4 Conclusion

Pair-balanced regression, odds calibration, and configuration-aware adaptation give a compact CPU predictor. On the held-out development base, calibration reduces own-label ALC by 0.004279 and peer-scenario ALC by 0.003479. Gains concentrate at budgets 0 and 1 and vary across tasks.

References

[1] AI Measurement Science. *The Predictive AI Evaluation Competition at NeurIPS 2026*. 2026. https://aims.stanford.edu/competition. Accessed October 31, 2026.

[2] Nhi Truong, Sang T. Truong, and Sanmi Koyejo. *The AI Measurement Data Bank*. AIMS Lab, Stanford University, 2026. https://huggingface.co/datasets/aims-foundations/measurement-db.

[3] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63:3–42, 2006. doi:10.1007/s10994-006-6226-1.

[4] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html.

A Detailed feature specification

The predictor constructs structured attributes and lexical features from the same normalized input pair.

A.1 Public subject identity and preprocessing

The eight identity fields are `normalized_name`, `provider`, `release_date`, `access_date`, `harness`, `harness_version`, `reasoning_effort`, and `subject_features_extra`. Normalization maps missing values and exact case-insensitive strings `none`, `null`, and `nan` to the empty string; other strings have surrounding whitespace stripped and retain their categorical case. The null-string check precedes whitespace stripping. Identity uses these complete normalized fields before predictive feature filtering. Two extra-configuration strings can therefore distinguish evidence groups while producing the same retained predictive attributes.

The predictive subject-extension whitelist is `model_config`, `source_model_name`, `released_variant`, `source_model_variant`, `model_identifier`, `source_model`, `source_model_identifier`, `defense`, `input_format`, `prompt_condition`, `instruction_size`, and `shot_size`. A number followed by B/b in the normalized model name supplies $\log(1 + \text{size})$ and a size-availability indicator. Absent matches yield zero for both.

A.2 Item attributes and handcrafted measurements

The item-extension whitelist is `lang`, `website`, `paper`, `competition`, `problem_idx`, `prompt_source`, `suite`, `context_length`, `task`, `category`, `context_condition`, `question_polarity`, `database_id`, `difficulty`, `prompt_condition`, and `capability`. Parsing accepts JSON dictionaries or semicolon-separated key–value pairs and serializes the retained entries. Non-dictionary JSON yields an empty extension.

The 22 handcrafted values comprise two length measurements, three character fractions, seven textual indicators, newline and brace counts, two ordinal descriptors, and six domain indicators. With retained text length L floored at one, the length measurements are $\log(1+L)/12$ and $\min(L/20000, 1)$. Digit, whitespace, and letter fractions, all seven binary indicators, newline count, and opening-brace count use the first 8,000 characters of the joined item text. Binary indicators detect mathematical markup, diff/traceback patterns, code, web actions, paper prompts, URLs, and code fences/diffs. Newline count is divided by 200, and opening-brace count is capped at 50 and divided by 50.

The six domains are code, math, web, document, research, and other. The detector first checks language, website, and paper attributes, then searches the first 4,000 characters of joined item text; the code-pattern expression alone uses the first 2,000 characters. Length measurements and length bins use the complete joined text after content truncation. These heuristic categories provide coarse task descriptors. Length-bin boundaries are 80, 300, 1,200, 5,000, and 20,000 characters. The domain index and length-bin index are each divided by five, and the six domain indicators give a one-hot encoding.

A.3 Exact branch inputs

The structured encoder receives the handcrafted values, retained categorical item attributes, nonempty public subject values after extension filtering, parsed model size, and reasoning-effort availability. Its fitted dictionary contains 1,444 columns. Ridge concatenates these with 65,536 hashed text columns, for 66,980 input dimensions; ExtraTrees uses the 1,444 structured columns. Hashing uses signed counts, lowercase word tokens of at least two characters, unigrams and bigrams, and L2 normalization.

Subject hashing joins normalized name, an empty `display_name`, provider, harness, harness version, reasoning effort, filtered extra attributes, and release date with `|` separators. The empty field remains between name and provider because the visible-subject whitelist omits `display_name`. Access date enters structured features and identity. Item text joins content, retained attributes, and interactors with newlines. Content is first limited to 12,000 characters; handcrafted features use the joined text, while hashing joins subject text with its first 12,000 characters. Structured attributes are also retained separately.

B Prediction algorithm and numerical details

For target (s, i) and evidence $\mathcal{L}$, prediction follows these steps:

1. Load the frozen model if necessary and compute the clipped base mixture $p_0(s, i)$.
2. Apply the fixed intercept to the target probability and the evidence probabilities used below.
3. Select target-benchmark labels with the target's public configuration identity. Apply Equation 9 when this own-label group is nonempty; otherwise keep the calibrated target probability.
4. Check for other-configuration labels on the target benchmark. If present, fit Equation 10 to all same-benchmark labels, with calibrated logits as fixed offsets.
5. Clip the own-label prediction to $[0.001, 0.999]$, then mix it with the hierarchical prediction with equal weights when sharing is active.
6. Clip the result to $[0.001, 0.999]$ and return a native float.

The cached state consists of model weights and deterministic feature values. Evidence groups are constructed from the current call's labels. A target with peer labels and no own labels receives a shared task adjustment; missing benchmark IDs lead to the base-only path.

B.1 Hierarchical Newton system

Let X have a leading column of ones and one indicator column per configuration represented by the evidence or target. Set $\theta = (u, v_1, \ldots, v_K)^\top$ and $\Lambda = \mathrm{diag}(0.5, 2, \ldots, 2)$. With $\pi = \sigma(z + X\theta)$, the gradient and Hessian are

$$g = X^\top(\pi - y) + \Lambda\theta, \qquad H = X^\top \mathrm{diag}(\pi_j(1 - \pi_j))X + \Lambda. \tag{13}$$

The code solves $H\Delta = g$ and updates $\theta \leftarrow \theta - \mathrm{clip}(\Delta, -2, 2)$ coordinate-wise. Positive diagonal regularization makes the Hessian positive definite in exact arithmetic even with redundant task/configuration columns. A target configuration with no observations has $\hat{v}_s = 0$: its likelihood column is zero, and its penalty is minimized at zero. This yields a direct dense solve for the point estimate.

With M labels and K configuration coordinates, a dense iteration costs approximately $O(MK^2 + K^3)$. The small label budget keeps the response side compact, while the number of configurations sets the system dimension. Ridge uses sparse features, and tree inference scales with the tree count and depth.

B.2 Pseudo-count anchoring and shrinkage

For $n > 0$, the updated mean m combines the acquisition mean probability and empirical label mean with empirical weight $n/(n + 2)$. The implementation clips inputs to logits, uses m directly in the constant branch, and clips the final output. Away from clipping bounds, the shifted probability is

$$r(p) = \frac{kp}{1 - p + kp}, \qquad k = \frac{m/(1 - m)}{\bar{p}/(1 - \bar{p})}, \qquad q(p) = \tfrac{1}{2}r(p) + \tfrac{1}{2}m. \tag{14}$$

Here $r(\bar{p}) = m$, so the acquisition mean is an anchor. Also $r'(p) = k/(1 - p + kp)^2 > 0$, preserving item ordering within an evidence group. Mixing with m halves any pairwise shifted difference: $q(p_1) - q(p_2) = \frac{1}{2}[r(p_1) - r(p_2)]$. The update therefore separates the success-level estimate from the retained item-level variation.

C Calibration candidate comparison

The candidates form the Cartesian product of intercept or slope/intercept calibration, penalty 0.001 or 0.01, and application at all budgets or only without benchmark evidence. A slope/intercept candidate fits $\sigma(a\,\mathrm{logit}(p) + b)$ with $a \in [0.1, 3]$, $b \in [-3, 3]$, and penalty $\lambda[(a - 1)^2 + b^2]$. The intercept candidate fixes $a = 1$ and uses λb^2. All candidates fit cold-start response Brier loss, then are compared by the complete adaptation ALC. In the offline `cold_only` candidates, only the zero-budget curve entry is replaced; positive-budget entries retain the uncalibrated curve. The selected all-budget candidate instead calibrates both target and evidence predictions before adaptation.

Selection accepts non-degrading ALC in both evidence scenarios and mean improvement exceeding 0.0002, then minimizes mean scenario ALC. The following table shows that applying calibration throughout adaptation improves on application only without evidence for both families and penalty strengths.

Table 6: Calibration candidate comparison on the development episodes.

Kind	Penalty	Application	Own ALC	Peer ALC	Mean ALC
Intercept	0.001	Without evidence	0.194306	0.192033	0.193170
Intercept	0.001	All budgets	0.191748	0.190275	0.191012
Intercept	0.01	Without evidence	0.194416	0.192142	0.193279
Intercept	0.01	All budgets	0.192035	0.190513	0.191274
Slope/intercept	0.001	Without evidence	0.194808	0.192534	0.193671
Slope/intercept	0.001	All budgets	0.193066	0.191399	0.192233
Slope/intercept	0.01	Without evidence	0.194629	0.192356	0.193492
Slope/intercept	0.01	All budgets	0.192607	0.190996	0.191802

C.1 Intercept objective and derivative

For episode/response weight $w_{ej} = 1/(EN_e)$, let $r_{ej} = \sigma(\text{logit}(p_{0,ej}) + b)$ and $p_{ej} = C(r_{ej})$. The derivative used by L-BFGS-B is

$$\frac{dL}{db} = 2\sum_{e,j} w_{ej}(p_{ej} - y_{ej})r_{ej}(1 - r_{ej})\mathbf{1}_{\{0.001<r_{ej}<0.999\}} + 2\lambda b. \tag{15}$$

The indicator implements the derivative of output clipping. The regularizer pulls the correction toward the identity transform $b = 0$. Starting at zero, the selected fit reaches $b = -0.5796672067792444$ with $\lambda = 0.001$. Equation 7 gives its fitting objective and Table 3 gives the data used at each stage.

C.2 Saved episode provenance and calibration assignment

`source/scripts/run_cal_v10.py:development()` concatenates three saved pools: `experiments/empirical-v7-20261028/development.joblib`, `experiments/empirical-v7-20261028/confirmation.joblib`, and `experiments/train-v9-20261029/confirmation.joblib`. These retain their historical seeds, item caps, and group-selection rules; the 698-episode development result is computed on the saved prediction cache, rather than a single newly generated split. The builder shuffles sorted unique item IDs with a group-specific generator derived from the seed, benchmark, and subject. It splits at the integer half, caps the evaluation half, samples one trial from each of the first 31 acquisition items, and retains all trials for the selected evaluation items. Historical construction code includes seed 0 with cap 80 for the original development tasks and seed 901029 with cap 60 and at most 16 groups per benchmark for the V9 confirmation pool.

For V10 confirmation, `prepare()` uses seed 1001029 and cap 60, then selects at most 12 groups per benchmark without replacement using the same seed. Counts in the listed benchmark order are 11, 12, 10, 12, 3, and 12. The 437 selection episodes span 36 benchmarks. Each of these benchmarks receives an intercept fitted after removing its episodes from that pool. The five original tasks contribute 1, 83, 120, 26, and 31 episodes respectively (261 total); each receives the shared intercept $b = -0.4100424649186898$ fitted to all 437 selection episodes. This is also the frozen confirmation intercept. Their exclusion applies to this intercept selection stage, not to all earlier V3 development.

Ridge tolerance 10^{-4} and ExtraTrees `max_features=1.0` are effective scikit-learn 1.7.2 defaults, not explicit arguments at the training call. The exported wrapper has `cold_only=False`, even though the class constructor defaults to `True`.

D Complete public development results

The tables give development and confirmation results for the two evidence scenarios. Own denotes target-configuration labels; Peer adds one same-benchmark subject group. Development combines

leave-one-benchmark-out calibration for 36 tasks with the shared 437-episode intercept for five original tasks; confirmation uses that frozen intercept, following Table 3.

D.1 Aggregate validation and uncertainty

Table 7: Public validation ALC. Development uses the 36-task leave-out/five-task shared-intercept protocol in Section 5; confirmation uses the frozen intercept.

Stage	Global calibration	Own labels	Additional peer
Development	None	0.196028	0.193754
Development	Benchmark-excluded intercept	**0.191748**	**0.190275**
Confirmation	None	0.177177	0.172908
Confirmation	Frozen intercept	**0.174028**	**0.170539**

Table 8: Confirmation ALC differences and 95% benchmark-cluster bootstrap intervals. Negative values favor calibration. Scenario average is the mean of the own-label and peer differences.

Scenario	Point difference	95% interval
Own labels	-0.003149	$[-0.016279,\ 0.008222]$
Additional peer	-0.002369	$[-0.012995,\ 0.006723]$
Scenario average	-0.002759	$[-0.014867,\ 0.007473]$

D.2 Per-budget development and confirmation curves

Table 9: Per-budget development and confirmation Brier scores.

Phase	Scenario	B_0	B_1	B_3	B_7	B_{15}	B_{31}
Dev. base	Own	0.252865	0.219054	0.197178	0.181258	0.172199	0.168037
Dev. base	Peer	0.252865	0.212976	0.192981	0.179882	0.172279	0.168442
Dev. cal.	Own	0.235650	0.210813	0.194096	0.180184	0.171859	0.167931
Dev. cal.	Peer	0.235650	0.206979	0.191018	0.179270	0.172092	0.168385
Conf. base	Own	0.269164	0.208435	0.172072	0.157569	0.144485	0.137482
Conf. base	Peer	0.269164	0.195768	0.166611	0.154044	0.144593	0.137887
Conf. cal.	Own	0.257644	0.203674	0.168635	0.156159	0.144193	0.137314
Conf. cal.	Peer	0.257644	0.192392	0.164795	0.153294	0.144490	0.137805

D.3 Development results by benchmark

Each row averages the episodes belonging to one benchmark. The episode-count column gives its weight in the primary aggregate: with benchmark counts E_b, the overall ALC is $\sum_b E_b A_b / \sum_b E_b$.

Table 10: Development ALC by benchmark and evidence scenario.

Benchmark	Episodes	Own base	Own cal.	Peer base	Peer cal.
abstract_reason	1	0.279652	0.258622	0.279652	0.258622
aegis	1	0.255484	0.249973	0.255484	0.249973
appworld	16	0.199162	0.188856	0.197564	0.189304
babilong	83	0.222807	0.222618	0.221727	0.221655
bertaqa	6	0.213746	0.216612	0.211536	0.213563
bird_sql	120	0.114161	0.098920	0.109723	0.097144
brace	12	0.146163	0.157494	0.144080	0.153350
causal_axioms	2	0.214420	0.224055	0.210533	0.217760
clasheval	6	0.252186	0.241558	0.244398	0.235767
cmmlu	20	0.191142	0.197277	0.191428	0.197004

Table 10: Development ALC by benchmark and evidence scenario (continued).

Benchmark	Episodes	Own base	Own cal.	Peer base	Peer cal.
cruxeval	20	0.251722	0.244524	0.250249	0.244174
devbench	16	0.219185	0.219974	0.236210	0.236548
disco	4	0.156943	0.136189	0.142748	0.126238
egoschema	11	0.265252	0.267356	0.264487	0.266426
emoji_attack	10	0.220741	0.206854	0.223050	0.211829
financebench	16	0.187711	0.183047	0.188419	0.185196
harmbench	20	0.235662	0.233726	0.235424	0.233929
healthadminbench	7	0.228193	0.213151	0.226746	0.213645
helm_bbq	16	0.122097	0.138990	0.119683	0.134035
helm_thaiexam	20	0.251318	0.248136	0.249563	0.247218
igakuqa	5	0.210270	0.211112	0.216038	0.217122
kormedmcqa	6	0.166469	0.177701	0.164110	0.173152
legal_rag_bench	6	0.162476	0.169100	0.161462	0.166994
legalbench	20	0.264352	0.263926	0.265287	0.265040
liveaopsbench	13	0.231143	0.220160	0.225111	0.216335
medhelm_pubmed_qa	13	0.221545	0.229075	0.218655	0.224344
mind2web	1	0.121562	0.143075	0.121562	0.143075
mmlu	16	0.244696	0.240339	0.243347	0.240351
mmoral_bench	12	0.263405	0.268236	0.260308	0.263700
multi_swebench	20	0.189548	0.173628	0.181342	0.168197
naep_llm_students	11	0.207116	0.224288	0.207817	0.223118
planbench	20	0.188147	0.174794	0.183241	0.172262
real_webagents	26	0.208557	0.196938	0.207829	0.197565
researchcodebench	31	0.223491	0.224596	0.220818	0.221771
safeagentbench	6	0.266896	0.259011	0.266718	0.259627
software_world_models	16	0.255969	0.258886	0.254252	0.256317
swe_rebench	1	0.268368	0.277640	0.268368	0.277640
truthfulqa_mc	20	0.256138	0.241401	0.251599	0.239107
wikihow_agent	12	0.081026	0.107501	0.074951	0.097087
wmt_mqm	16	0.133876	0.112702	0.118854	0.101498
workbench_revisited	20	0.187206	0.190388	0.185244	0.188400

D.4 Fresh confirmation results by benchmark

Table 11: Confirmation ALC by benchmark and evidence scenario.

Benchmark	Episodes	Own base	Own cal.	Peer base	Peer cal.
ceval	11	0.274343	0.264790	0.266177	0.259782
helm_bold	12	0.105294	0.078077	0.088679	0.066324
jailbreakbench	10	0.134986	0.144203	0.142933	0.150442
mathconstraint	12	0.220075	0.222660	0.218527	0.220449
medhelm_med_qa	12	0.126305	0.139335	0.124718	0.135588
swe_smith	3	0.280966	0.268704	0.278046	0.267333

D.5 Task changes and weighted contributions

For development benchmark b, let E_b denote its episode count and $d_b = A_b^{\text{cal}} - A_b^{\text{base}}$. Its contribution to the aggregate change is $(E_b/698)d_b$. Figure 2 aligns both quantities on a shared benchmark axis; parentheses give episode counts.

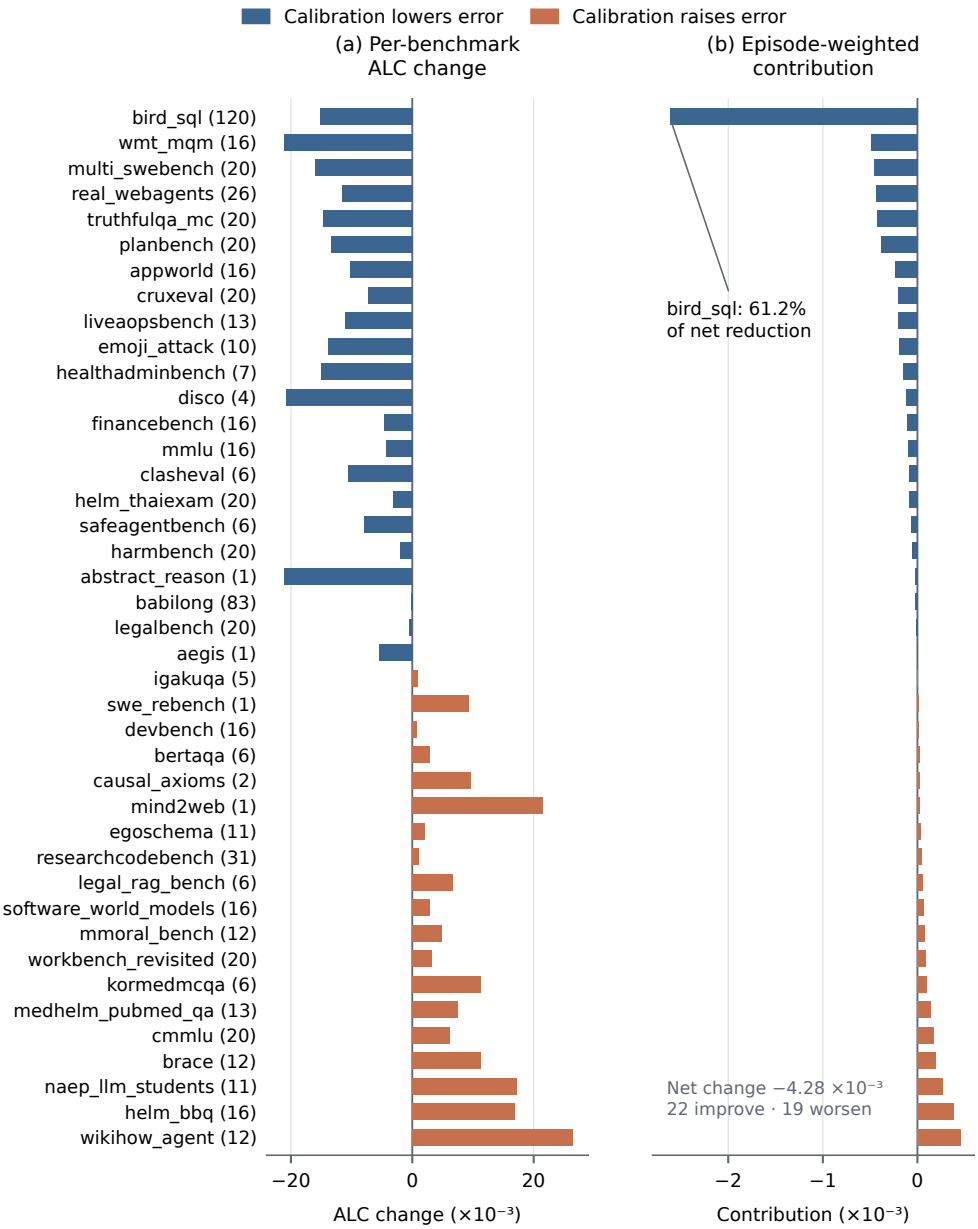


Figure 2: Own-label development results for all 41 benchmarks. Left: calibrated minus uncalibrated ALC. Right: episode-weighted contributions $(E_b/698)d_b$. Both panels share the task axis and use units of 10^{-3}. Calibration lowers error on 22 benchmarks and raises it on 19; `bird_sql` supplies 61.2% of the net reduction. Bars use the six-decimal ALCs in Table 10.

E Online record transcription

The nine V10 formative records are listed in their recorded order. Their mean is 0.184069, median 0.189484, sample standard deviation 0.026420, and range 0.139514–0.218715.

Table 12: Nine recorded V10 formative submissions in recorded order.

Index	Codabench submission ID	Brier ALC
1	1018663	0.156288
2	1020184	0.201272
3	1020550	0.139514
4	1020563	0.218715
5	1020962	0.166623
6	1021081	0.189484
7	1021083	0.176881
8	1020950	0.212794
9	1020922	0.195047

F The 105 final training benchmarks

Candidate counts give the retained response pool before 150,000-row sampling. Each listed benchmark passes the public-release and binary-response filters. The total is 55,047,908 candidate responses across 105 tasks.

Table 13: Candidate responses for the 105 final training benchmarks.

Public benchmark	Candidate responses
abstract_reason	9,095
advprompter	100
aegis	11,608
afrieval	1,403,380
agentic_review_perturbation	315
agi_elo	277,106
aider	5,356
algotune	2,772
appworld	10,530
auditing_sabotage_bench	1,869
autoresearchbench	115
averimatec	152
babilong	2,020,951
beavertails	1,680
bertaqa	57,072
bird_sql	43,200
brace	202,096
bridging_gap	4,706,881
causal_axioms	9,000
cellverse	748
ceval	14,806
chatgpt_drift	46,832
clasheval	75,216
cmmlu	254,804
confagents	64
critic_discernment_game	7,276
cruxeval	992,000
dave	19,248
dbpa	409
devbench	5,149
disco	111,906
editbench	23,328
edumath	16,720
egoschema	5,500

Table 13: Candidate responses for the 105 final training benchmarks (continued).

Public benchmark	Candidate responses
ehrflowbench	140
elicitation_game	14,070
emoji_attack	58,860
enginemt_qa	676
felm	4,426
fewshot_ttt_bbh	7,425
financebench	2,400
haiid	71,364
harmbench	107,274
healthadminbench	7,020
helm_afr	745,360
helm_bbq	81,000
helm_bold	42,000
helm_thaiexam	23,730
icpc2_code_selector	70,905
igakuqa	7,355
igakuqa119	400
ikp	268,351
jailbreakbench	3,074
judgetuning	21,000
kormedmcqa	21,391
legal_rag_bench	1,800
legalbench	63,457
lingoly	24,926
live_agent_risk	256
liveaopsbench	69,264
llm_bp_tag	158,596
llm_uncertainty_bench	5,100,000
mathconstraint	11,862
medhelm_ehr_sql	13,000
medhelm_head_qa	13,000
medhelm_med_mcqa	13,000
medhelm_med_qa	13,000
medhelm_medbullets	4,004
medhelm_medcalc_bench	13,000
medhelm_medec	7,761
medhelm_medhallu	13,000
medhelm_pubmed_qa	11,000
medhelm_race_based_med	2,171
mind2web	17,153
mmlu	2,106,300
mmoral_bench	28,679
monkey_power_laws	28,780,000
multi_swebench	57,975
mypcbench	118
naep_llm_students	53,680
nester	1,128
ood_prediction	142,805
pku_saferlhf	164,236
planbench	38,061
preference_dissection	167,680
proofaug	244
prox	24,298
real_webagents	4,449
researchcodebench	6,784
revolve	3,100
safeagentbench	4,500
software_world_models	7,830
sugarcrepe	15,024
swe_rebench	67,074
swe_smith	49,897

Table 13: Candidate responses for the 105 final training benchmarks (continued).

Public benchmark	Candidate responses
tensortrust	681,726
truthfulqa_mc	122,550
tulu_human_eval	2,300
visual_riddles	3,186
visualwebarena	600
weavebench	114
wikihow_agent	171,412
wmt_mqm	4,879,368
workarena	475
workbench_revisited	16,560
Total	55,047,908

G Reproduction workflow

Run these commands from the root of `V10_1020550_code_bundle` with the dependencies in Section 8. Original-weight inference and cache-based calibration fitting use the bundled local files.

The recorded environment is Python 3.10.21, NumPy 2.2.6, SciPy 1.15.3, scikit-learn 1.7.2, joblib 1.6.0, and threadpoolctl 3.7.0. Training-table processing additionally uses pandas 2.3.3 and pyarrow 25.0.1. The runtime and training dependency lists are `requirements_runtime.txt` and `requirements_training_reference.txt` in the bundle root.

G.1 Integrity and original-weight inference

```
python tools/verify_bundle.py
python tools/check_submission.py
```

The first command compares file hashes and the extracted runtime bytes against the original ZIP members. The second loads the model in isolation and runs the interface checks. For direct integration, add `runtime` to Python's module path, import `predict` from `model`, and pass a public subject/item pair and the current evidence list.

G.2 Cache-based calibration refit

```
python tools/fit_selected_calibration.py
```

Inputs are `calibration_cache/development_predictions.joblib` and `calibration_cache/confirmation_predictions.joblib`, containing public-task predictions and labels. The script fits Equation 7 and checks agreement with the final intercept to 10^{-10}. It reports the 758-episode count, 47 public benchmarks, fitted intercept, and penalized objective. Caching isolates this inexpensive fitting operation from feature extraction and regression.

G.3 Base retraining

```
python tools/train_selected_base.py \
  --raw-data /path/to/pinned/measurement-db \
  --output-dir outputs/retrained_v10 \
  --sample-out outputs/final_sample_150k.parquet
```

The raw-data entry checks the 105-task set and 55,047,908 candidate rows, samples with seed 0, joins selected features, and trains trees with seed 42. The default numerical thread budget is four. Outputs include sampled row identifiers, the base and wrapped models, and training metadata in the requested new directory. A saved feature-attached sample can instead be passed with `-sample-in`. Sampling and snapshot requirements are discussed in Section 8.

The historical entry point `source/scripts/run_cal_v10.py` implements held-out base fitting, prediction caching, calibration search, confirmation, and export. Its inputs include training samples,

development/confirmation episode pools, and validation weights under the experiments tree. The supplied public prediction caches provide inputs to the final intercept fit. Preserving the earlier episode and training artifacts alongside these caches would connect the full sequence into a replayable workflow.

G.4 Submission packaging

```
python tools/package_submission.py
python tools/check_submission.py \
  outputs/submission_v10_repacked.zip
```

The packager collects the entry point, fitted weights, training metadata, six runtime modules, and dependency declaration. It writes a new ZIP and compares its members with the original archive. Separating runtime packaging from training and calibration materials keeps the scoring package compact.

By default, packaging reads `runtime/` and therefore repackages the bundled original weights. To package a retrained model, first copy that runtime tree to a new directory, replace its `artifacts/ensemble.joblib` and `artifacts/training.json` with the corresponding files from the retraining output, and pass the new directory with `-runtime`. The comparison then lists those changed members; the isolated interface check accepts the new ZIP path as above.

H Artifact index

Table 14: Runtime, training, and evaluation artifacts.

Artifact	Contents
`runtime/artifacts/training.json`	Final calibration parameters, objective, and confirmation results.
`evidence/base_training_metadata.json`	Base sample size, task names, recipe, and filtering audit.
`evidence/selection_historical.json`	Development calibration scores and candidate search.
`evidence/confirmation_historical.json`	Confirmation scores and cluster-bootstrap intervals.
`evidence/selected_budget_curve.csv`	Submission 1020550's six aggregate budget errors.
`evidence/same_version_submissions.csv`	IDs and scores for nine V10 formative submissions.
`evidence/package_check.json`	Isolated interface checks and recorded timing.
`evidence/runtime_source_parity.json`	Hash comparisons of source and runtime modules.

Original archive SHA-256:

`4d1d1ada9d9e6ac74b39676e6c75647f6bc5136a299b09a62839e12e814c53af`

I Data and code provenance

The Measurement Data Bank curation contributions use CC BY-SA 4.0; individual benchmark materials retain their upstream licenses and terms [2].

The existing V3 codebase supplies the base ensemble recipe and the beta2 and hierarchical adaptation implementations, retained in `ensemble_v2.py`, `adaptive_v3.py`, and `shared_v3.py`. V10 reuses these components and adds the global calibration wrapper in `calibrated_v10.py`, together with held-out calibration selection, intercept refitting, and submission export in `source/scripts/run_cal_v10.py`. The training recipe is recorded as `senior_v3_frozen_w0_beta2` in the base-model metadata. The artifact index identifies the associated source, fitted-state, and evaluation records.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Sections 5–7 identify the estimator stage for each result and report the task-level distribution of effects.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 8 discusses representation, calibration transfer, peer-context coverage, and training reconstruction.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This empirical report provides algorithm definitions and elementary algebraic identities.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 2–8 disclose the data processing, model, calibration, and evaluation protocols. Appendix G provides original-weight inference, fixed-recipe training, and cache-based calibration commands. The released weights and prediction caches support verification; Section 8 states the limits on byte-identical retraining.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The public repository linked in Section 8 provides code and reproduction materials, with commands and dependency versions in Appendix G. The public Measurement Data Bank source is cited in Section 2.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Sections 2–5 specify filtering, weights, features, model settings, capped episode construction, saved development-pool provenance, the 36-task/five-task calibration assignment, and candidate selection; Appendix C.2 records seeds and group caps.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Section 6 defines the benchmark-cluster bootstrap and its percentile intervals. Section 7 reports the mean, median, standard deviation, and range of the nine formative records.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: Section 8 gives CPU threading and algorithmic costs; historical machine specifications and complete training-resource measurements are unavailable.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The study uses public measurement tables and competition-provided labels, with data provenance in Appendix I and experimental limitations in Section 8.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [No]

Justification: The report focuses on predictive-evaluation methodology and experimental analysis.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The new fitted asset is a compact probability regressor trained on public measurement tables.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: Appendix I documents dataset terms and inherited V3 components. A complete per-source creator and code-license mapping is not provided.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.

- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The code bundle documents model parameters, task lists, dependency versions, interface checks, and checksums, with the file index and commands in the appendices.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The experiments use existing AI-system response measurements.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The experiments use existing AI-system response measurements.

    Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: The predictor consists of regression ensembles, lexical hashing, and probability updates. Section 8 discloses the use of OpenAI Codex for code implementation, experiment execution, validation, and documentation.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.